



МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА - Российский технологический университет»
РТУ МИРЭА

Институт Информационных Технологий
Кафедра Вычислительной Техники

ПРАКТИЧЕСКАЯ РАБОТА №3

по дисциплине
«Проектирование интеллектуальных систем (часть 1/2)»

Студенты группы: ИКБО-43-23

Чумаков С.Ю., Хайров Э.Н.
(Ф. И.О. студента)

Преподаватель

Холмогоров В.В.
(Ф.И.О. преподавателя)

Москва 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 ПОСТАНОВКА ЗАДАЧИ	4
2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	5
2.1 Алгоритм KNN	5
2.2 Метрики классификации	6
2.3 Дерево решений.....	8
2.4 Случайный лес.....	9
3 ДОКУМЕНТАЦИЯ К ДАННЫМ.....	11
3.1 Описание предметной области	11
3.2 Анализ данных.....	11
3.3 Предобработка данных	16
4 ПРАКТИЧЕСКАЯ ЧАСТЬ	19
4.1 Алгоритм KNN	19
4.2 Дерево решений.....	23
4.3 Случайный лес.....	24
ЗАКЛЮЧЕНИЕ	27
СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ	29
ПРИЛОЖЕНИЯ.....	30

ВВЕДЕНИЕ

Современные интеллектуальные информационные системы активно применяются в задачах анализа данных, автоматизации принятия решений и построения прогностических моделей. Одним из важнейших направлений обработки информации является задача классификации, заключающаяся в отнесении объектов к заранее известным категориям на основании признаков.

Классификация используется в самых различных предметных областях: от медицинской диагностики и финансового скоринга до биоинформатики и анализа потребительского поведения. Особое место занимает применение классификации в финансовой индустрии для обнаружения мошеннических транзакций, где своевременное выявление аномальных операций позволяет минимизировать убытки и защитить клиентов банков.

Эффективность алгоритма классификации зависит от качества предварительной обработки данных, выбора информативных признаков, устойчивости модели к дисбалансу классов (когда мошеннических транзакций значительно меньше, чем нормальных), а также от адекватности выбранного алгоритма.

В рамках данной работы проводится анализ применимости различных алгоритмов классификации для решения задачи бинарного распознавания мошеннических финансовых транзакций. Для построения и оценки моделей используются как стандартные средства библиотек машинного обучения, так и собственные реализации алгоритмов. Также выполняется расчёт ключевых метрик качества, позволяющих оценить точность, полноту и общую эффективность построенной модели классификатора.

1 ПОСТАНОВКА ЗАДАЧИ

Цель работы: приобрести навыки классификации как инструмента категориального и предикативного анализа данных с контролируемым обучением.

Задачи: определить предметную область решаемой задачи, найти или сгенерировать набор данных для выбранной задачи, проведя предварительную предобработку и подготовку данных, выбрать модель классификации, провести её обучение и тестирование, определить качество модели с помощью метрик потерь, изучить алгоритмы классификации, написать программный код для реализации указанных алгоритмов, провести бинарную и/или многоклассовую классификацию данных подготовленного набора.

2 ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

2.1 Алгоритм KNN

Алгоритм k-ближайших соседей (k-nearest neighbors, KNN) относится к числу простых и интуитивно понятных методов классификации. Его суть заключается в том, что для нового объекта определяется класс на основе классов ближайших к нему объектов из обучающей выборки. Алгоритм не требует этапа обучения в традиционном смысле и относится к ленивым (instance-based) методам обучения.

Классификация объекта происходит по большинству голосов среди k ближайших соседей, найденных по выбранной метрике расстояния. При этом может применяться взвешивание голосов: более близким объектам присваивается больший вес, чем дальним.

Алгоритм KNN включает следующие шаги:

1. Выбор параметра k – количества ближайших соседей.
2. Вычисление расстояний от классифицируемого объекта до всех объектов обучающей выборки. Обычно используется одна из метрик:
 - Евклидова метрика (Формула 2.1.1):

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (2.1.1)$$

- Манхэттенская метрика (Формула 2.1.2):

$$d(x, y) = \sum_{i=1}^n |x_i - y_i| \quad (2.1.2)$$

- Косинусная мера (Формула 2.1.3):

$$d(x, y) = 1 - \frac{x * y}{\|x\| * \|y\|} \quad (2.1.3)$$

3. Выбор k ближайших объектов (с минимальным расстоянием).
4. Определение метки класса. Если используется равное голосование (uniform), побеждает класс, наиболее часто встречающийся среди соседей. Если используется взвешенное голосование (distance), вес каждого соседа обратно пропорционален расстоянию (Формула 2.1.4).

$$w_i = \frac{1}{d(x, x_i) + \varepsilon}, \quad (2.1.4)$$

где ε – малое число, чтобы избежать деления на ноль.

5. Присвоение метки класса объекту на основе голосов.

Применение в задаче обнаружения мошенничества: KNN эффективен при обнаружении мошенничества, так как мошеннические транзакции часто образуют характерные кластеры в пространстве признаков (например, обнуление счёта + крупная сумма). Однако при использовании его на практике необходимо тщательно подбирать значение параметра k , а также учитывать важность масштабирования и балансировки классов из-за сильного дисбаланса (мошенников 0.13% против 99.87% нормальных транзакций).

2.2 Метрики классификации

Для оценки качества работы классификационных алгоритмов применяются различные метрики качества, отражающие, насколько точно и надёжно модель предсказывает метки классов. В данной работе рассматриваются следующие основные метрики: accuracy, precision, recall, F1-мера, а также матрица ошибок (confusion matrix).

1. Accuracy (доля правильных предсказаний)

Метрика accuracy (точность в смысле «правильности» предсказаний) отражает долю правильно классифицированных объектов от общего числа наблюдений (Формула 2.2.1).

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}, \quad (2.2.1)$$

где TP – объекты, правильно отнесённые к положительному классу;

TN – объекты, правильно отнесённые к отрицательному классу;

FP – объекты, ошибочно отнесённые к положительному классу;

FN – объекты, ошибочно отнесённые к отрицательному классу.

Метрика эффективна при сбалансированных классах, но может вводить в заблуждение при дисбалансе.

2. Precision (Точность)

Метрика precision показывает, какую долю предсказанных положительных объектов действительно составляют положительные примеры (Формула 2.2.2).

$$Precision = \frac{TP}{TP+FP} \quad (2.2.2)$$

Высокое значение precision означает, что среди объектов, отнесённых моделью к положительному классу, большинство действительно принадлежат к этому классу.

3. Recall (полнота)

Метрика recall отражает долю правильно предсказанных положительных объектов среди всех реально положительных (Формула 2.2.3).

$$Recall = \frac{TP}{TP+FN} \quad (2.2.3)$$

Высокая полнота указывает на то, что модель пропускает мало объектов положительного класса.

4. F1-Score (F-мера)

F1-Score — гармоническое среднее между Precision и Recall, позволяющее оценить баланс между ними (Формула 2.2.4).

$$F1 = 2 * \frac{Precision * Recall}{Precision + Recall} \quad (2.2.4)$$

Значение F1 варьируется от 0 до 1, где 1 соответствует идеальной классификации.

5. Confusion matrix (матрица ошибок)

Матрица ошибок (confusion matrix) позволяет визуально отразить, как модель путает классы. Это квадратная таблица размером $N * N$, где N — число классов. Строки представляют истинные метки (диагональные элементы), а столбцы — предсказанные.

2.3 Дерево решений

Дерево решений (Decision Tree) — это один из простейших и наиболее интерпретируемых алгоритмов классификации и регрессии. Его основная идея заключается в построении древовидной структуры, в узлах которой выполняются проверки по значениям признаков, а в листьях — находятся метки классов или числовые прогнозы.

Алгоритм рекурсивно разбивает обучающее пространство на подпространства, основываясь на признаках, которые наилучшим образом разделяют данные. Такое разбиение продолжается до тех пор, пока не будут достигнуты листья дерева, содержащие итоговые решения (классы).

Внутренние узлы содержат логические правила вида $x_j \leq t$, где x_j — значения j -ого признака, а t — порог. Ветви представляют возможные исходы проверки условия. Листья содержат предсказанные метки классов.

Шаги алгоритма:

1. Выбирается наилучший признак и порог для разбиения текущей выборки.
2. Выбор осуществляется по метрике качества.
3. Данные разбиваются на две подгруппы.
4. Для каждой из подгрупп повторяется процесс рекурсивно.

5. Процесс завершается, если достигнуты максимальная глубина, минимальное количество объектов в узле, все объекты в узле принадлежат одному классу.

Для выбора оптимального признака и порога разделения используются метрики, минимизирующие неоднородность данных:

1. Энтропия — мера хаоса в данных (Формула 2.3.1).

$$\text{Энтропия}(S) = - \sum_{i=1}^C p_i \log_2 p_i, \quad (2.3.1)$$

где p_i — доля объектов класса i в подмножестве S ;

C — число классов.

2. Индекс Джини — мера неоднородности (Формула 2.3.2).

$$\text{Джини}(S) = 1 - \sum_{i=1}^C p_i^2 \quad (2.3.2)$$

2.4 Случайный лес

Случайный лес (Random Forest) — это ансамблевый метод машинного обучения, основанный на построении множества решающих деревьев и агрегации их предсказаний. Он относится к классу бэггинг-методов (bagging), которые направлены на уменьшение переобучения и повышение обобщающей способности модели.

Случайный лес создаёт множество деревьев решений, каждое из которых обучается на случайной подвыборке исходных данных. При этом в каждом узле дерева для разделения выбирается случайное подмножество признаков. Итоговое решение принимается на основе голосования деревьев (в задаче классификации) или усреднения (в задаче регрессии).

Шаги алгоритма:

1. Для каждого дерева случайным образом выбирается подмножество объектов из обучающей выборки с возвращением (bootstrapping). При

построении дерева в каждом узле выбирается случайное подмножество признаков для нахождения лучшего разделения.

2. Шаг 1 повторяется для заданного числа деревьев n .
3. Предсказание на основе голосования большинства деревьев (Формула 2.4.1).

$$y = \operatorname{argmax}_c \sum_{t=1}^T I(h_t(x) = c) , \quad (2.4.1)$$

где $h_t(x)$ – предсказание t -го дерева;
 T – число деревьев.

3 ДОКУМЕНТАЦИЯ К ДАННЫМ

3.1 Описание предметной области

В качестве набора данных выбран синтетический датасет PaySim для обнаружения мошеннических финансовых транзакций, доступный на платформе Kaggle

PaySim – это симулятор мобильных денежных переводов, созданный на основе реальных транзакций банка в Африке. Датасет содержит синтетические записи о финансовых операциях клиентов за период в 30 дней (744 часа). Целью является построение модели классификации, способной автоматически выявлять мошеннические транзакции на основе характеристик операции.

Такие модели используются банками и платёжными системами в режиме реального времени для блокировки подозрительных операций, защиты клиентов от кражи средств и минимизации финансовых потерь. Своевременное обнаружение мошенничества позволяет предотвратить перевод денег на счета мошенников и заморозить подозрительные аккаунты.

3.2 Анализ данных

В исходных данных представлено **6,362,620 транзакций**. Каждая запись соответствует конкретной финансовой операции одного из пяти типов. Датасет содержит **11 столбцов** (10 признаков и целевая метка isFraud). Колонки с признаками включают:

1. step: Единица времени (1 шаг = 1 час), всего 744 шага (30 дней).
2. type: Тип транзакции: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER.
3. amount: Сумма транзакции в местной валюте.
4. nameOrig: Идентификатор клиента-отправителя.
5. oldbalanceOrg: Баланс отправителя ДО транзакции.

6. newbalanceOrig: Баланс отправителя ПОСЛЕ транзакции.
7. nameDest: Идентификатор получателя.
8. oldbalanceDest: Баланс получателя ДО транзакции
9. newbalanceDest: Баланс получателя ПОСЛЕ транзакции
10. isFraud: Целевая переменная – 1 = мошенничество, 0 = нормальная транзакция
11. isFlaggedFraud: Флаг системы безопасности (помечает транзакции > 200,000 как подозрительные)

Распределение транзакций по классам отображено на Рисунке 3.2.1.

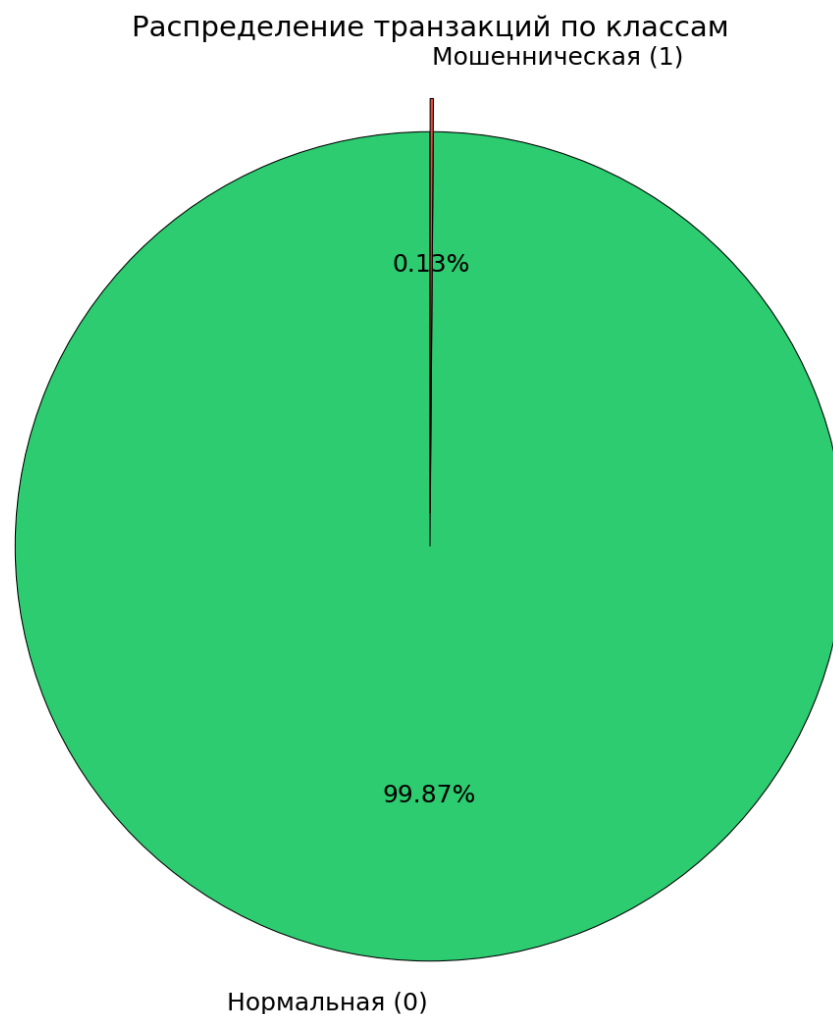


Рисунок 3.2.1 – Распределение транзакций по классам

Доля мошеннических транзакций по типу операции представлена на Рисунке 3.2.2.

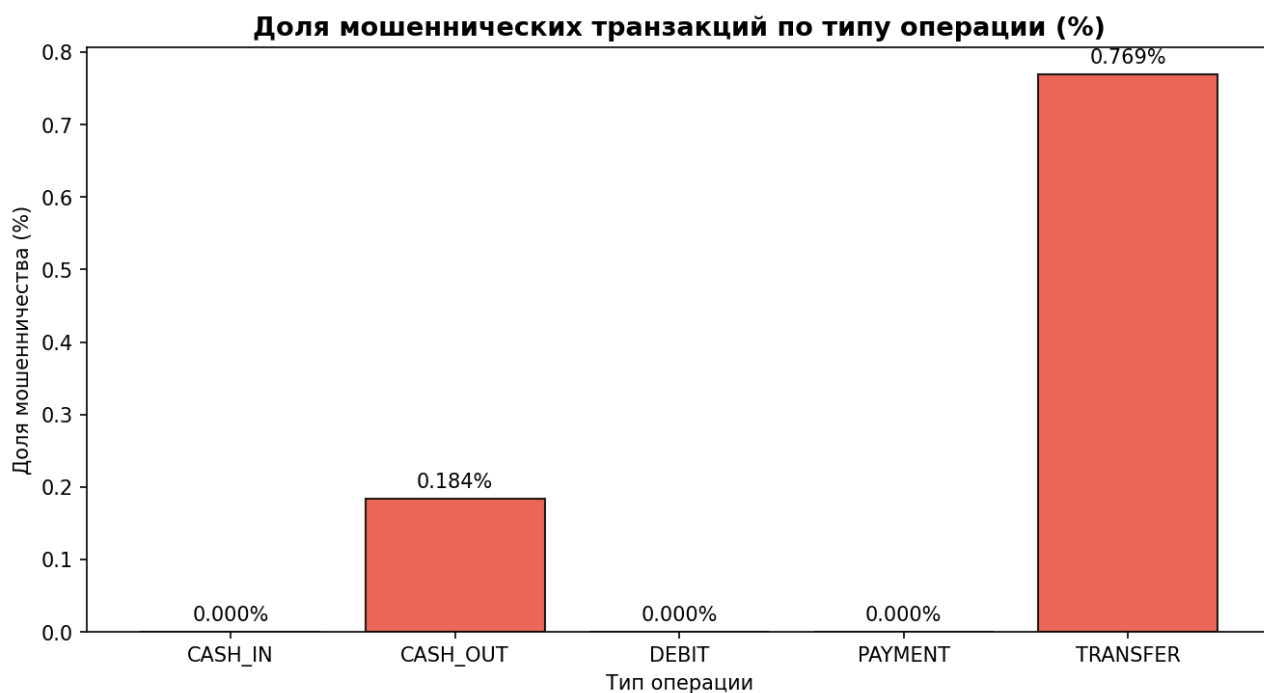


Рисунок 3.2.2 – Доля мошеннических транзакций по типу операции

Мошенничество встречается только в транзакциях типов CASH_OUT и TRANSFER. Это означает, что признак type — один из самых информативных для классификации. Все транзакции типов CASH_IN, DEBIT и PAYMENT — гарантированно нормальные.

Гистограммы числовых признаков представлены на Рисунке 3.2.3.

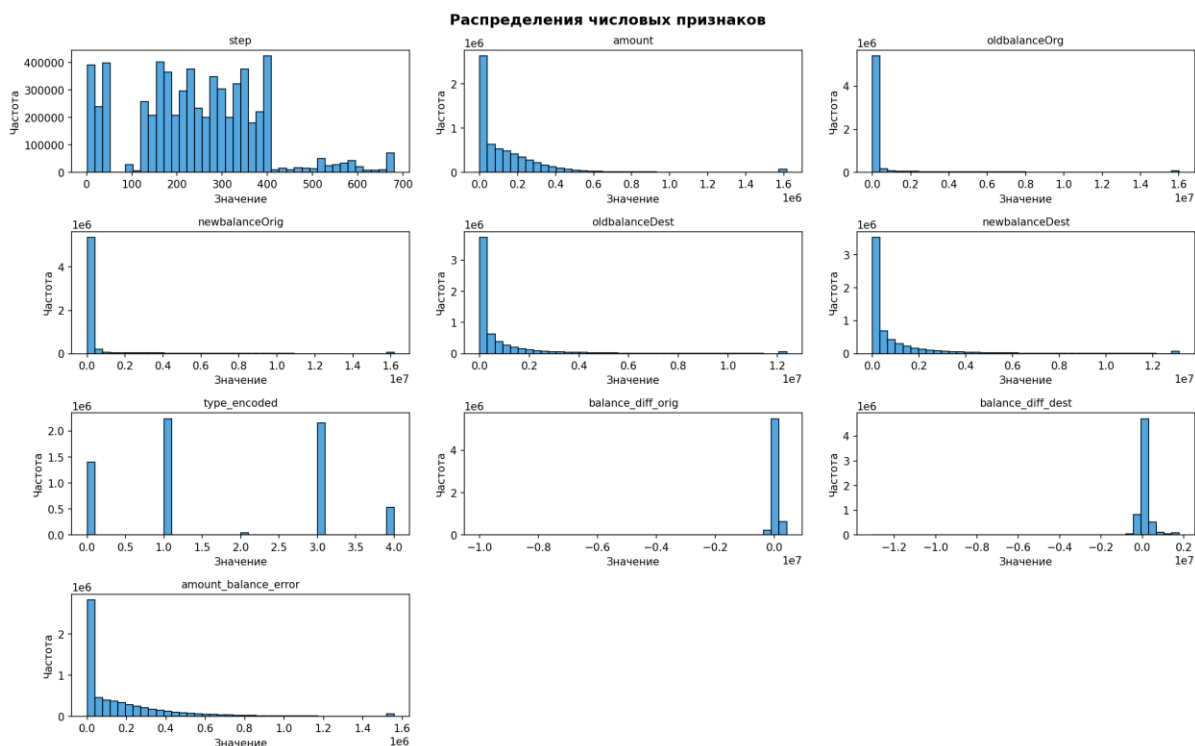


Рисунок 3.2.3 - Гистограммы распределений признаков

Большинство числовых признаков имеют правостороннюю скошенность – основная масса значений сосредоточена в области малых величин, а хвост распределения вытянут вправо. Это характерно для финансовых данных: большинство транзакций совершается на небольшие суммы, тогда как крупные переводы встречаются значительно реже.

Признаки `oldbalanceOrg`, `newbalanceOrig`, `oldbalanceDest` и `newbalanceDest` имеют схожий характер распределения – большая часть значений равна нулю (счета с нулевым балансом), а остальные сосредоточены в диапазоне малых величин с редкими крупными значениями.

Признак `amount` также скошен вправо: преобладают небольшие транзакции, крупные суммы встречаются редко.

Признак `step` распределён относительно равномерно в диапазоне от 1 до 743, что соответствует равномерному потоку транзакций на протяжении 30 дней.

Подобное распределение признаков подтверждает необходимость масштабирования данных перед обучением моделей, поскольку признаки имеют существенно различающиеся диапазоны значений.

Матрица рассеяния признаков представлена на Рисунке 3.2.4.

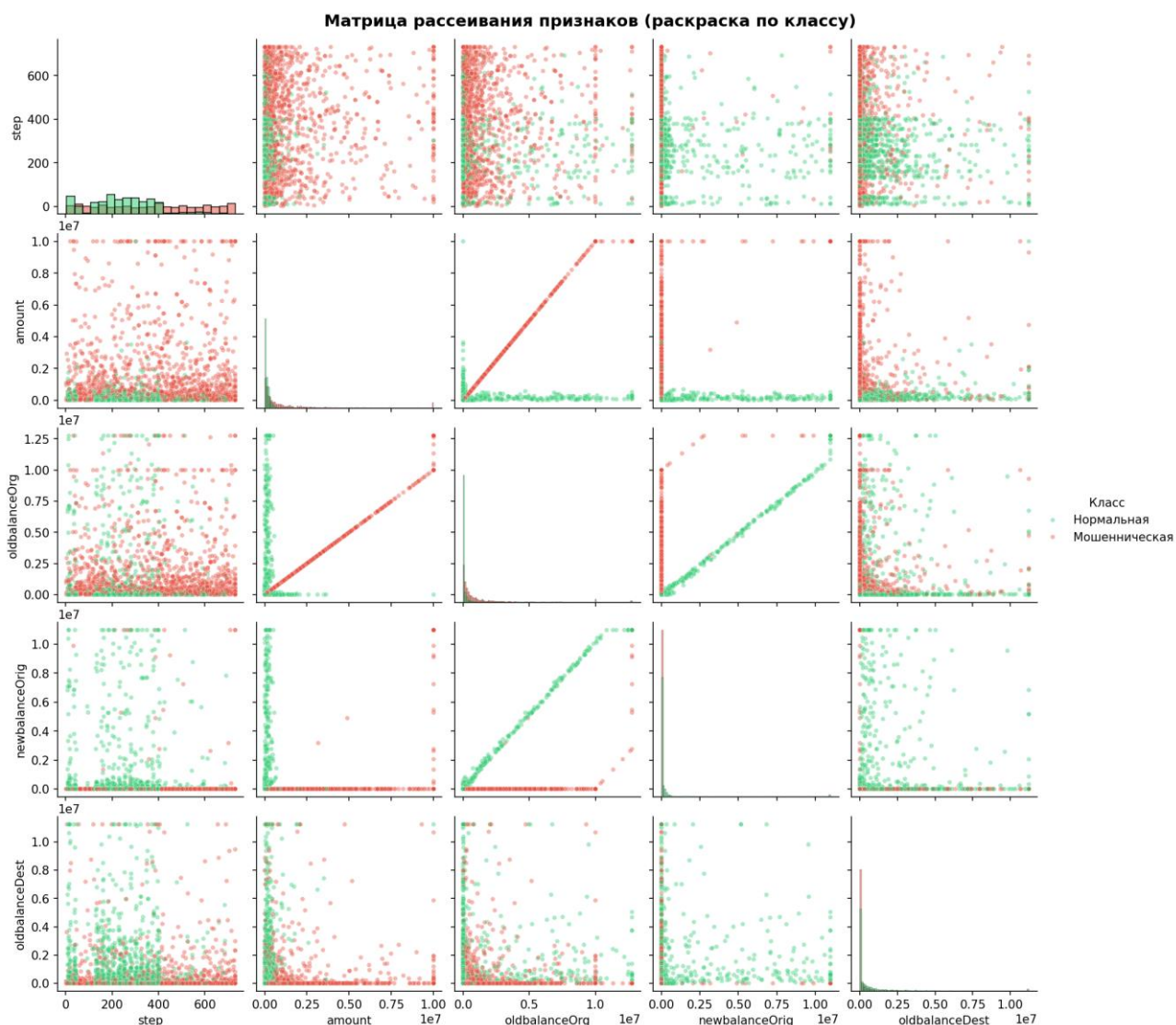


Рисунок 3.2.4 – Матрица рассеивания признаков

На матрице рассеивания видно

Чёткое разделение между классами (зелёные точки — нормальные транзакции, красные — мошеннические) в пространстве признаков `amount` и `balance_diff_orig`.

Мошеннические транзакции образуют характерные кластеры: обнуление счёта отправителя (`newbalanceOrig = 0`) при крупной сумме перевода.

Между признаками прослеживается корреляция, что говорит о возможности создания новых комбинированных признаков для улучшения качества классификации.

Матрица корреляции признаков представлена на Рисунке 3.2.5.

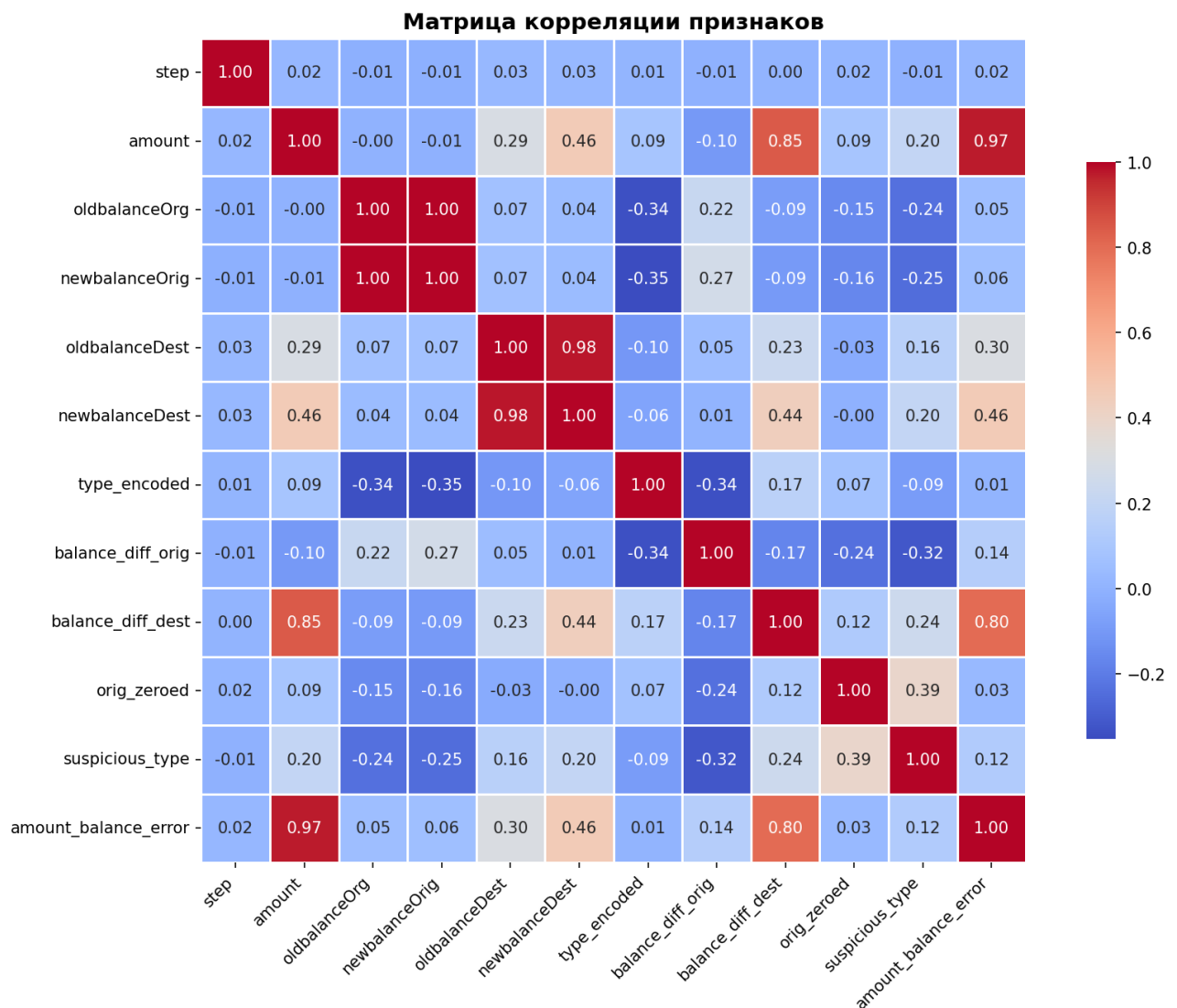


Рисунок 3.2.5 – Матрица корреляции признаков

Матрица корреляции показывает:

Сильная корреляция (>0.9) между oldbalanceOrg и newbalanceOrig — логично, так как новый баланс = старый - сумма транзакции.

Умеренная корреляция между amount и balance_diff_orig, что ожидаемо.

Признак isFraud имеет заметную корреляцию с инженерными признаками (orig_zeroed, amount_balance_error), что подтверждает их информативность.

3.3 Предобработка данных

Реализованы следующие этапы предобработки данных:

- удаление дубликатов строк;
- проверка пропущенных значений;

- удаление нечисловых значений;
- масштабирование признаков.

Удаление дубликатов строк подразумевает обнаружение и удаление полностью идентичных записей (строк) в исходном датасете. Под «идентичностью» понимается совпадение всех значений по всем признакам. В рассматриваемом датасете дубликатов не обнаружено.

Удаление нечисловых значений — столбцы `nameOrig`, `nameDest` (строковые идентификаторы) и `isFlaggedFraud` (дублирует часть информации) удалены, так как не несут численной информации для моделей.

Масштабирование признаков (`StandardScaler`) — приведение всех числовых признаков к единому масштабу. `StandardScaler` — стандартизация признаков.

`StandardScaler` — один из наиболее распространённых способов стандартизации. Для каждого признака j вычисляется среднее μ_j и стандартное отклонение σ_j . Каждое значение x_{ij} преобразуется по Формуле 3.3.1.

$$x'_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \quad (3.3.1)$$

В результате стандартизированный признак x'_{ij} имеет среднее 0 и стандартное отклонение 1.

Данные разделены на обучающую и тестовую выборки в соотношении 80:20. Обучающая выборка содержит 5,090,096 образца, а тестовая выборка — 1,272,524 образца.

Без стратификации при случайном разбиении может возникнуть ситуация, когда в тестовой выборке окажется слишком мало (или слишком много) мошенников, что исказит оценку качества модели.

Балансировка классов (`SMOTE`) — синтетическая генерация примеров миноритарного класса. `SMOTE` (`Synthetic Minority Over-sampling Technique`) создаёт синтетические примеры мошенников путём интерполяции между

существующими мошенническими транзакциями. В данной работе используется параметр `sampling_strategy=0.1`, что означает целевое соотношение мошенников к нормальным транзакциям 1:10 (вместо исходного 1:773).

Значения ДО и ПОСЛЕ балансировки представлены на Рисунке 3.2.6.

```
Предобработка завершена. Итого: 6,362,077 строк.
Данные разделены (стратифицировано, 80/20):
  Обучение:      5,089,661 строк (мошенников: 6,558, 0.129%)
  Тестирование: 1,272,416 строк (мошенников: 1,639, 0.129%)

До балансировки (SMOTE):
  Нормальных:    5,083,103
  Мошеннических: 6,558

После балансировки (SMOTE):
  Нормальных:    5,083,103
  Мошеннических: 508,310 (9.09%)
```

Рисунок 3.2.6 – Значения ДО и ПОСЛЕ балансировки

SMOTE позволяет улучшить Recall (модель находит больше мошенников), но может незначительно снизить Precision (больше ложных тревог). Лучше проверить лишнюю транзакцию вручную, чем пропустить реального мошенника.

Предобработанная обучающая выборка представлена на рисунке 3.2.7

```
Первые строки предобработанной обучающей выборки:
step    amount  oldbalanceOrig  newbalanceOrig  oldbalanceDest  newbalanceDest  type_encoded  balance_diff_orig  balance_diff_dest  orig_zeroed  suspicious_type  amount_balance_error
0  0.566284 -0.294343 -0.273787 -0.278439 -0.323829 -0.333423 0.952472 -0.160561 -0.152900 -0.560418 -0.878257 -0.331495
1  0.587361 0.162588 -0.288725 -0.292455 -0.137211 -0.085118 -0.528874 -0.145749 0.189063 -0.560418 1.138619 0.126673
2  0.418742 0.510856 -0.288725 -0.292455 1.433527 1.425321 -0.528874 -0.145749 0.447782 -0.560418 1.138619 0.473309
3  0.292277 -0.297311 -0.282991 -0.286917 -0.323829 -0.333423 0.952472 -0.148272 -0.152900 -0.560418 -0.878257 -0.331495
4  0.376587 -0.294784 -0.220111 -0.224035 -0.305447 -0.316932 -1.269548 -0.132764 -0.155230 -0.560418 -0.878257 -0.325252
5  0.741929 -0.134568 -0.281699 -0.251791 -0.322743 -0.333423 -1.269548 0.530440 -0.157443 -0.560418 -0.878257 -0.006393
6 -0.628104 -0.274824 -0.288622 -0.292455 0.181197 0.137609 -0.528874 -0.147785 -0.135745 1.784382 1.138619 -0.309000
7  0.917574 -0.205322 -0.288725 -0.292455 -0.323829 -0.333423 0.952472 -0.145749 -0.152900 -0.560418 -0.878257 -0.239351
8 -0.621078 -0.297186 -0.197856 -0.202852 -0.323829 -0.333423 0.952472 -0.148792 -0.152900 -0.560418 -0.878257 -0.331495
9  1.065116 -0.117555 1.433147 1.445549 -0.244838 -0.289977 -1.269548 0.600863 -0.286858 -0.560418 -0.878257 0.027465
```

Рисунок 3.2.7 – Предобработанная обучающая выборка

На данном этапе полученный очищенный набор признаков может использоваться в алгоритмах классификации.

4 ПРАКТИЧЕСКАЯ ЧАСТЬ

4.1 Алгоритм KNN

Для решения задачи классификации реализован алгоритм К-ближайших соседей с использованием библиотеки `scikit-learn`. Алгоритм написан в файле `KNN.py`, содержание которого представлено в Приложении Б.

В рамках эксперимента выбрано значение параметра $k = 5$, а стратегия взвешивания соседей — «distance», то есть чем ближе сосед, тем больший вес он имеет при голосовании.

После обучения модели на обучающих данных, сделаны предсказания на тестовой выборке. По результатам вычислены метрики (Таблица 4.1.1).

Таблица 4.1.1 – Метрики по результатам классификации с библиотечной реализацией

Метрика	Значение	Интерпретация
accuracy	0.9986	Модель правильно классифицировала 99.86% транзакций из тестовой выборки
precision	0.4756	47.56% транзакций, помеченных как мошеннические, действительно являются мошенническими
recall	0.8902	Модель нашла 89.02% реальных мошенников (пропустила только ~11% мошенников)
f1	0.6199	Гармоническое среднее между Precision и Recall. Умеренное значение из-за разрыва между Precision и Recall
roc_auc	0.9581	95.81% — модель уверенно разделяет классы, высокая способность отличить мошенников от нормальных транзакций

Также построена матрица ошибок (confusion matrix) (Рисунок 4.1.1).



Рисунок 4.1.1 – Матрица ошибок по результатам классификации алгоритмом KNN

Модель пропустила всего 180 мошенников из 1639, что составляет Recall = 89.02%. При этом 1609 нормальных транзакций ошибочно помечены как мошеннические. Это приемлемо для системы первичной фильтрации, где подозрительные транзакции отправляются на ручную проверку.

На рисунке 4.1.2 представлена ROC кривая.

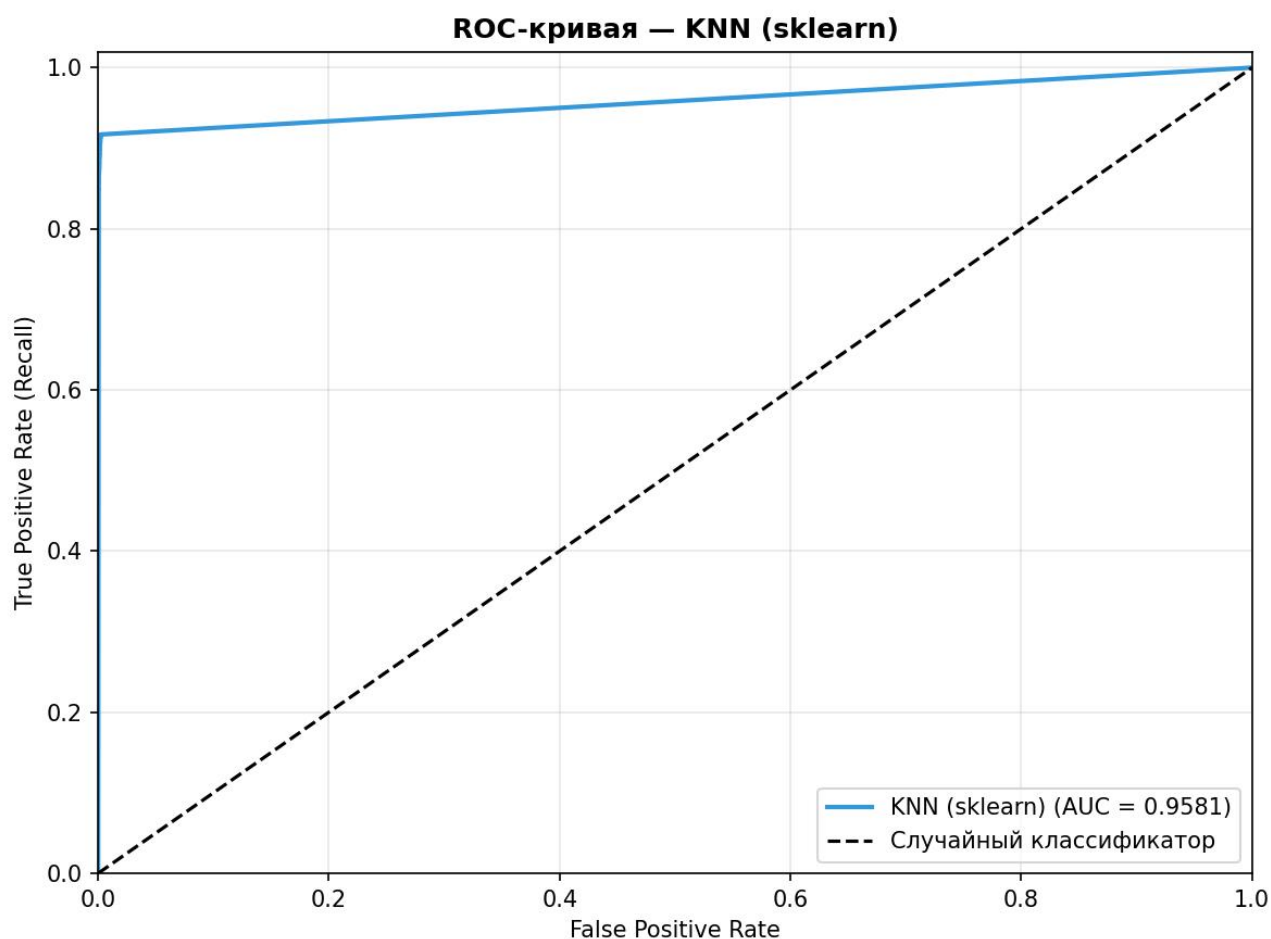


Рисунок 4.1.2 – ROC-кривая

ROC-кривая показывает зависимость True Positive Rate (Recall) от False Positive Rate при различных порогах классификации. Площадь под кривой (AUC) = 0.9581 означает, что модель с вероятностью 95.81% правильно упорядочит случайную пару «мошенник — нормальная транзакция».

Для демонстрации принципов работы алгоритма KNN также разработана его самостоятельная реализация на языке Python без использования библиотек машинного обучения. Код написан в файле KNN_custom.py, содержание которого представлено в Приложении В.

для демонстрации используется подвыборка:

- Обучающая: 5,000 записей
- Тестовая: 1,000 записей

После обучения модели на обучающей выборке и предсказания меток для тестовых данных вычислены метрики качества (Таблица 4.1.2).

Таблица 4.1.2 – Метрики по результатам классификации с собственной реализацией

Метрика	Значение	Интерпретация
accuracy	0.9740	Модель правильно классифицировала 97.40% объектов подвыборки
precision	0.9665	96.65% помеченных транзакций действительно являются мошенническими
recall	0.9820	Модель нашла 98.20% реальных мошенников в подвыборке
f1	0.9742	Отличный баланс между Precision и Recall, модель корректно обнаруживает мошенничество

Собственная реализация показывает **высокое качество классификации** на стратифицированной подвыборке (Accuracy = 97.40%, F1 = 97.42%). Матрица ошибок подтверждает корректность работы алгоритма: из 500 мошенников пропущено только 9, из 500 нормальных ошибочно помечено 17. Результаты сопоставимы с библиотечной реализацией sklearn, что подтверждает правильность собственной реализации алгоритма. Высокие метрики на подвыборке 50/50 объясняются сбалансированным распределением классов — на полном датасете с дисбалансом 773:1 без балансировки результаты были бы ниже.

Матрица ошибок для самописной реализации представлена на Рисунке 4.1.2.



Рисунок 4.1.2 - Матрица ошибок по результатам классификации алгоритмом KNN с собственной реализацией

4.2 Дерево решений

Дополнительно реализована и протестирована модель классификации на основе дерева решений с использованием библиотеки `scikit-learn`. Классификатор написан в файле `DecisionTree.py`, содержание которого представлено в Приложении Г.

В качестве критерия расщепления использовался индекс Джини, а глубина дерева = 10, данное ограничение нужно для предотвращения переобучения.

После обучения выполнена классификация тестовой выборки. Полученные метрики представлены в Таблице 4.2.1.

Таблица 4.2.1 – Метрики по результатам классификации с библиотечной реализацией Decision tree

Метрика	Значение	Интерпретация
accuracy	0.9999	Модель правильно классифицировала 99.99% транзакций
precision	0.8975	89.75% помеченных транзакций действительно мошеннические
recall	0.9988	Модель нашла 99.88% реальных мошенников (пропустила всего 0.12%)
f1	0.9866	Почти идеальный баланс между Precision и Recall
roc_auc	0.9994	99.94% — практически идеальное разделение классов

Также с помощью встроенной функции plot_tree() выполнена визуализация структуры обученного дерева, позволяющая наглядно увидеть правила принятия решений и важность признаков (Рисунок 4.2.1).

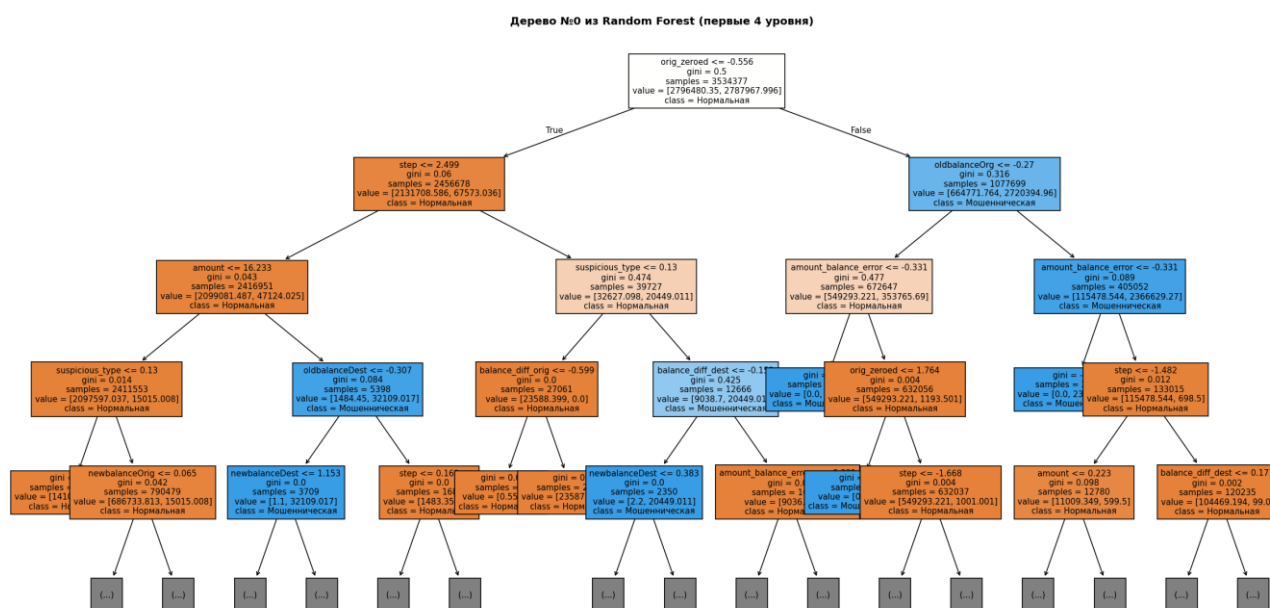


Рисунок 4.2.1 – Визуализация дерева принятия решений

4.3 Случайный лес

В завершении реализована модель классификации на основе алгоритма случайного леса с использованием библиотеки sklearn.

Модель написана в файле RandomForest.py, содержание которого представлено в Приложении Д.

Для построения случайного леса использовались следующие параметры:

- количество деревьев (n_estimators) – 100;
- критерий разбиения – индекс Джини (gini);
- глубина деревьев = 15;
- количество признаков для выбора при расщеплении – sqrt.

После обучения модели проведена оценка качества классификации с использованием основных метрик (Таблица 4.3.1).

Таблица 4.3.1 – Метрики по результатам классификации с библиотечной реализацией Random Forest

Метрика	Значение	Интерпретация
accuracy	1.0000	Модель правильно классифицировала 100% транзакций из тестовой выборки
precision	0.9675	96.75% помеченных транзакций действительно являются мошенническими
recall	0.9982	Модель нашла 99.82% реальных мошенников (пропустила всего 3 из 1,639)
f1	0.9826	Почти идеальный баланс между Precision и Recall

Random Forest показывает наилучшие результаты среди всех протестированных моделей. Это обусловлено:

- Снижением переобучения за счёт усреднения предсказаний 100 деревьев
- Способностью находить сложные нелинейные зависимости
- Устойчивостью к выбросам и шуму благодаря ансамблю

Random Forest нашёл 99.82% мошенников (пропустил всего 3 из 1,639) при Precision = 96.75% (всего 55 ложных тревог из 1,270,777 нормальных транзакций). Это production-ready результат для системы обнаружения мошенничества.

Дополнительно выполнена оценка важности признаков с использованием встроенной функции `feature_importances_`. Распределение важности представлено в Таблице 4.3.2.

Таблица 4.3.2 – Распределение важности признаков

Признак	Важность
amount_balance_error	0.2554
balance_diff_orig	0.2512
orig_zeroed	0.1707
suspicious_type	0.1013
oldbalanceOrg	0.0627
newbalanceOrig	0.0518
amount	0.0384
balance_diff_dest	0.0231
type_encoded	0.0158
step	0.0122
newbalanceDest	0.0116
oldbalanceDest	0.0058

В отличие от одного дерева решений, где признак `amount_balance_error` доминировал с важностью 66.66%, в Random Forest важность более равномерно распределена между несколькими признаками. Это говорит о том, что ансамбль учитывает комбинации признаков и не полагается на единственный фактор. Тройка лидеров: `amount_balance_error` (25.5%), `balance_diff_orig` (25.1%) и `orig_zeroed` (17.1%) — все три непосредственно отражают характерные паттерны мошенничества: обнуление счёта жертвы и манипуляции с балансами.

ЗАКЛЮЧЕНИЕ

В рамках практической работы была решена задача бинарной классификации мошеннических финансовых транзакций на датасете PaySim, содержащем более 6,3 миллиона записей. Ключевой особенностью датасета является критический дисбаланс классов — соотношение 773:1, при котором стандартная метрика Accuracy оказывается бесполезной. По этой причине основными метриками оценки служили Precision, Recall и F1-score, а для работы с дисбалансом применялись SMOTE и параметр `class_weight='balanced'`.

Наилучшие результаты показал алгоритм случайного леса ($F1 = 98.26\%$, $ROC-AUC = 99.96\%$), пропустивший всего 3 мошенника из 1,639 при 55 ложных тревогах. Высокое качество объясняется природой ансамблевого метода: усреднение предсказаний 100 независимых деревьев снижает переобучение и повышает устойчивость к шуму в данных.

Дерево решений незначительно уступило по Precision ($F1 = 94.54\%$), выдав 187 ложных тревог против 55 у Random Forest, однако сохранило практически такой же Recall = 99.88%. Его преимуществом остаётся интерпретируемость — структура дерева наглядно показывает, какие признаки приводят к классификации транзакции как мошеннической.

Алгоритм KNN показал наиболее скромные результаты ($F1 = 61.99\%$), что объясняется его чувствительностью к дисбалансу классов: мошеннические транзакции в пространстве признаков окружены нормальными соседями, что затрудняет их уверенную идентификацию даже после балансировки. Собственная реализация KNN подтвердила корректность алгоритма на подвыборке ($F1 = 97.42\%$), однако непригодна для полного датасета из-за вычислительной сложности.

Таким образом, для задач обнаружения финансового мошенничества с сильным дисбалансом классов наиболее эффективны ансамблевые методы на основе деревьев решений. Random Forest рекомендуется в качестве основного

классификатора для автоматической фильтрации подозрительных транзакций в режиме реального времени.

СПИСОК ИНФОРМАЦИОННЫХ ИСТОЧНИКОВ

1. Сорокин, А. Б. Безусловная оптимизация. [Электронный ресурс] : учебно-метод. пособие / А. Б. Сорокин, О. В. Платонова, Л. М. Железняк — М. РТУ МИРЭА , 2020.
2. Сорокин, А. Б. Введение в генетические алгоритмы: теория, расчеты и приложения. [Электронный ресурс] : учебно-метод. пособие / А. Б. Сорокин — М. МИРЭА , 2018.
3. Метод К-ближайших соседей (KNN). Принцип работы, разновидности и реализация с нуля на Python [Электронный ресурс]: Habr. URL: <https://habr.com/ru/articles/801885/> (Дата обращения: 16.05.2025).
4. Машинное обучение: Классификация методом KNN. Теория и реализация. С нуля. На чистом Python [Электронный ресурс]: Habr. URL: <https://habr.com/ru/articles/866636/> (Дата обращения: 17.05.2025).
5. Бэггинг и случайный лес. Ключевые особенности и реализация с нуля на Python [Электронный ресурс]: Habr. URL: <https://habr.com/ru/articles/801161/> (Дата обращения: 19.05.2025).

ПРИЛОЖЕНИЯ

Приложение А — Файл `dataset_manager.py` для предобработки и анализа датасета.

Приложение Б — Файл `KNN.py` с использованием готовой реализации алгоритма KNN.

Приложение В — Файл `KNN_custom.py` с собственной реализацией алгоритма KNN.

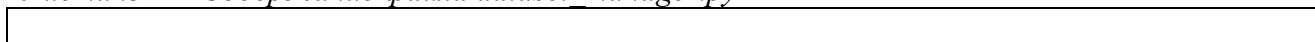
Приложение Г — Файл `DecisionTree.py` с использованием готовой реализации алгоритма Decision Tree.

Приложение Д — Файл `RandomForest.py` с использованием готовой реализации модели Random Forest.

Приложение А

Файл `dataset_manager.py` для предобработки и анализа датасета

Листинг А – Содержание файла `dataset_manager.py`



Приложение Б

Файл KNN.py с использованием готовой реализации алгоритма KNN

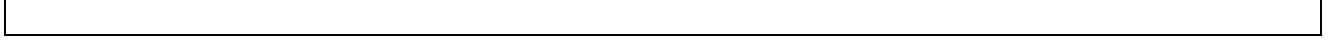
Листинг Б – Файл KNN.py

--

Приложение В

Файл KNN_custom.py с собственной реализацией алгоритма KNN

Листинг В – Файл KNN_custom.py



Приложение Г

Файл `DecisionTree.py` с использованием готовой реализации алгоритма `Decision Tree`

Листинг Г – Файл `DecisionTree.py`

--

Приложение Д

Файл RandomForest.py с использованием готовой реализации модели Random
Forest

Листинг Д – Файл RandomForest.py

--